Introduction to Machine Learning (67577)
Miluim Challenge:
Robust Image Classification

July 13, 2026

1 Motivation

Image classification is one of the most common and important tasks in machine learning. Given an image, the goal is to correctly predict its class. Modern image classification models can achieve impressive performance on clean validation sets, but they may also rely on misleading visual cues. For example, a goose photographed on water may be classified correctly as a goose, while the same goose photographed on grass or against an unusual background may be classified as another bird. In such cases, the model is not only recognizing the bird itself, but also relying too strongly on its background or typical habitat (see the related example in Figure 1).

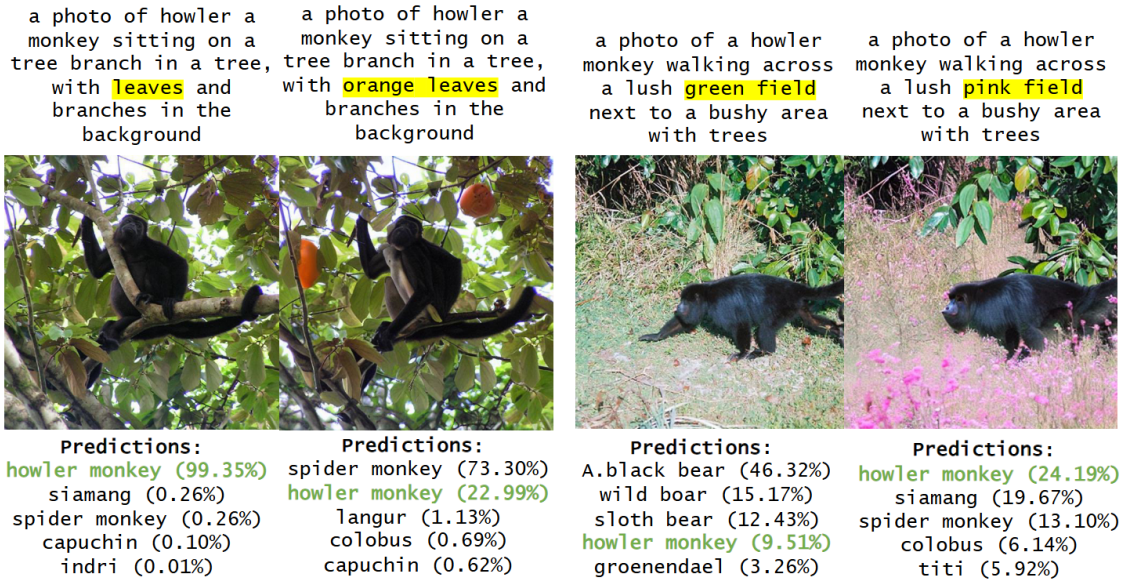

Figure 1: *Example from Prabhu et al. (2023), LANCE: Stress-testing Visual Models by Generating Language-guided Counterfactual Images. The authors show that changing the background/color context of a howler monkey image can change a ResNet-50 prediction from "howler monkey" to various other labels.*

In this challenge, you will be the R&D team for `PixelPerfect`, a startup company dedicated to

making accurate and robust bird-classification models. You will train an image classification model from scratch. Your goal is to classify images into 20 bird classes, *while making your model robust to visual manipulations*. In particular, your model should perform well both on regular validation images and on images with changes that should not affect the true bird label, such as shifts in background context, altered lighting conditions, or color distortions.

2 Tasks

In this challenge, you are required to submit a model trained from random weight initialization. Your core technical objective is to design a training strategy that balances standard bird-classification performance with visual invariance.

2.1 Standard Image Classification

In the first part of the challenge, your model should perform well on the given bird training images. Your first task is to split this data properly, choose an architecture and a loss function, and establish a working classification pipeline.

2.2 Robust Image Classification

In the second part of the challenge, you must make your model robust to visual manipulations. For example, changing the background of a bird image should ideally not change the predicted class if the bird itself remains the same. You will need to think critically about how to simulate these changes and where in your pipeline to apply them, for example during training or validation, to improve generalization against hidden, unseen augmentations.

3 Evaluation

The goal of your model is to achieve high bird-classification accuracy while being robust to visual manipulations. The final evaluation will test your submission on both in-distribution bird images, which are similar to the clean data you download, and out-of-distribution bird images created using held-out augmentations. The final score will combine your performance on regular bird images with your robustness to manipulated bird images.
You can assume that the test set will be evenly distributed between in-domain samples, which are similar to those given in the training set, and out-of-domain samples, which are bird images produced using held-out augmentations that are not available in the input data.

4 Data and Setup

To get started, first download the starter project files from Moodle. Then, download the folders `train_set` and `augmentations` from here. Place the downloaded dataset into the project directory so your setup looks like the tree below.

Starter Project Structure:

```
project/
├── dataset/ # Place raw image folders here
│   └── labels.json # Class-to-integer mapping
├── submissions/
│   ├── dummy_submission/ # Baseline example scripts
│   │   ├── model.py
│   │   ├── train.py
│   │   ├── predict.py
│   │   └── README
│   └── my_team/ # Contains templates for your code.  Start working here
├── base_model.py
├── labels.py
├── evaluate.py
└── check_submission.py
```

4.1 Project Files Overview

To help you orient yourself, here is a high-level overview of the provided starter files and directories:

- `dataset/`: This is where you will place the downloaded data folder. It contains 20 classes with exactly 1000 raw training samples allocated per class.
- `labels.json` dictates the explicit mapping sequence between textual class names and integer targets (0 to 19).
- `submissions/my_team/`: This is where you should start working. It contains placeholders for your code (`model.py`, `train.py`, and `predict.py`) as well as a `README`. You will modify these files and ultimately rename this folder to your team's IDs for your final submission.
- `dummy_submission/`: Contains the structural templates and baseline examples for the files you will eventually submit.
  - `model.py`: Where you will define your model architecture.
  - `train.py`: Where you will write your training loop to generate learned weights.
  - `predict.py`: The fixed interface that the grader will use to evaluate your model.
  - `README`: The template for your final experiment report.
- `base_model.py`: Defines the `BaseModel` class template that your `predict.py` inherits from. It also contains the `ImageNetSubset` dataset class, which you can import and use during your development process.
- `evaluate.py` & `check_submission.py`: Utility scripts to help you test locally. Both scripts are designed to iterate through and check *every* folder located inside the `submissions/` directory. `check_submission.py` verifies your structural constraints, while `evaluate.py` runs your inference loop and returns a final rank for your model.

5 Model Development Workflow

5.1 Recommended Modeling Pipeline

Here is a recommended pipeline:

1. **Download and Setup:** Download the `train_set` and place all images inside `dataset/`.
2. **Split the Data:** Before doing anything else, split the raw `train` data into a proper training set and a local validation set.

3. **Standard Training:** Design your model architecture and write a training script to optimize your network. Save your trained weights as a local artifact. Ensure you achieve good baseline accuracy on your clean, local validation set first.
4. **Brainstorm Manipulations:** Once your baseline works, start thinking about image manipulations. What visual changes might confuse your model?
5. **Integration Strategy:** Decide *how* and *where* to utilize these manipulations. Should you insert them directly into the training data? Should you create a separate stress-test validation set?
6. **Robust Training:** Implement your chosen robustness strategy, retrain your model, and evaluate if it maintains standard accuracy while resisting the visual manipulations you designed.

6 Submission Instructions

6.1 Implementation and Prediction Constraints

During evaluation, the submitted model receives a batch of preprocessed images formatted as a PyTorch tensor and must output one predicted class index per image:

$$\hat{y} = f_\theta(x), \qquad \hat{y}_i \in \{0, 1, \ldots, 19\}.$$

To ensure compatibility with our automatic grading system, your files must adhere to the following technical constraints:

model.py:

- Must define a class exactly named `ModelArchitecture`.
- `ModelArchitecture` must be importable from `predict.py`.
- Do not hide the architecture definition solely inside `train.py`. The evaluator must be able to reconstruct the model architecture directly through `model.py`.
- The architecture should output logits for the 20 target classes.

train.py:

- Must train `ModelArchitecture` using only the allowed training data.
- The trained parameters must be saved to exactly `weights.joblib`.
- To ensure hardware-independent loading during evaluation, save the model state dictionary after moving the model to CPU. For example:

```
state_dict = model.cpu().state_dict()
joblib.dump(state_dict, "weights.joblib")
```

Predict.py:

Must not be change

README:

- **Line 1:** Team members' names, separated by commas with no spaces.
- **Line 2:** Team members' IDs, separated by commas with no spaces.
- After an empty line, include a short description of your model design.
- You must elaborate on the manipulations you tried, your strategy for inserting them, and how they impacted performance.

Global Technical Restrictions:

- You are required to train your model **from scratch**.
- No external pretrained models or external datasets are allowed.
- No external API calls or third-party inference services are permitted.
- Your submission must not execute network download queries during training, loading, or evaluation.

6.2 Submission Folder Layout

Only one team member should submit the project on Moodle. Submit a folder named explicitly matching your student ID. Your team folder must be named `challenge2_IDs` (for example: `challenge2_123456789_234567891_345678912`).

What to Submit:

Your internal submission folder must follow this exact layout:

```
challenge2_IDs/
    train.py
    model.py
    predict.py
    weights.joblib
    README
```

Do Not Submit:

The dataset assets; active virtual environments; localized cache structures; intermediate checkpoints other than the required `weights.joblib`; or configurations containing absolute directory paths (e.g., `C:/Users/...`).

6.3 How to Test Yourself Locally

Before uploading your compressed run, test the folder directory structures locally using the evaluation checker provided in the starter files:

```
python check_submission.py
```

Note: You can also use the provided `evaluate.py` script to run a simulated local evaluation using your `predict.py` script.

6.4 Final Checklist

Before submitting, verify that:

- `weights.joblib` is saved inside your team directory;
- `model.py` correctly implements the standalone `ModelArchitecture` class layout;
- `predict.py` remains structurally unchanged from the provided evaluation wrapper interfaces;
- Your `README` thoroughly details the manipulation strategies you experimented with;
- Predictions return exact matched shapes bounded between 0 and 19;
- All system scripts operate safely without relying on static local computer absolute paths.

7 Interview.

As part of the final evaluation, each team will participate in a short interview. During the interview, you should be prepared to explain your modeling choices, describe the reasoning behind your design decisions, and walk the interviewer through your implementation and results. No presentation is required; the interview is intended to assess understanding and the ability to discuss the solution clearly.